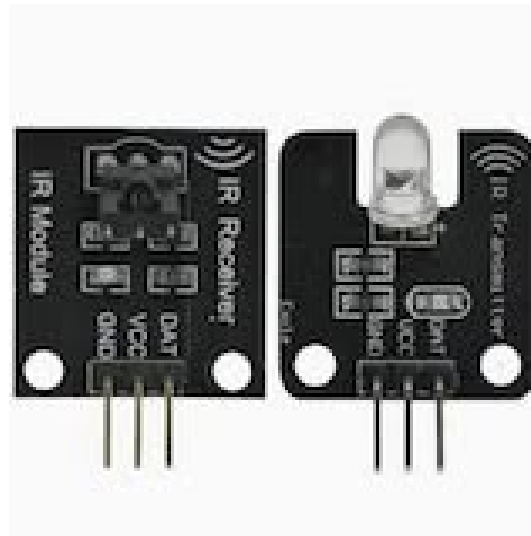


IrDA: Infrared Data Association Practice

IR Module

We can use a simple IR LED and phototransistor to send and receive infrared signals.



Basic IR Decoder

```
#include <IRremote.h>

#define IR_RECV_PIN 11  // IR receiver connected to pin 11

IRrecv irrecv(IR_RECV_PIN);
decode_results results;

void setup() {
  Serial.begin(9600);
  irrecv.enableIRIn();  // Start IR receiver
  Serial.println("IR Receiver Ready");
}
```

```

void loop() {
  if (irrecv.decode(&results)) {
    // Print received code
    Serial.print("Code: 0x");
    Serial.print(results.value, HEX);
    Serial.print(" ");
    Serial.print(results.bits);
    Serial.print(" bits");

    // Print protocol
    Serial.print(" Protocol: ");
    switch (results.decode_type) {
      case NEC:
        Serial.println("NEC");
        break;
      case SONY:
        Serial.println("Sony SIRC");
        break;
      case RC5:
        Serial.println("RC5");
        break;
      case RC6:
        Serial.println("RC6");
        break;
      default:
        Serial.println("Unknown");
    }

    irrecv.resume(); // Ready for next code
  }
}

```

Control LED with TV Remote

```
#include <IRremote.h>

#define IR_RECV_PIN 11
#define LED_PIN 13

IRrecv irrecv(IR_RECV_PIN);
decode_results results;

// Define remote button codes (learn these first!)
#define BUTTON_POWER 0xFF629D
#define BUTTON_VOL_UP 0xFF02FD
#define BUTTON_VOL_DOWN 0xFF9867

bool ledState = false;

void setup() {
  Serial.begin(9600);
  pinMode(LED_PIN, OUTPUT);
  irrecv.enableIRIn();
}
```

```

void loop() {
  if (irrecv.decode(&results)) {
    unsigned long code = results.value;

    // Ignore repeat codes (0xFFFFFFFF)
    if (code != 0xFFFFFFFF) {
      switch (code) {
        case BUTTON_POWER:
          ledState = !ledState;
          digitalWrite(LED_PIN, ledState);
          Serial.println(ledState ? "LED ON" : "LED OFF");
          break;

        case BUTTON_VOL_UP:
          Serial.println("Volume UP");
          // Control something else
          break;

        case BUTTON_VOL_DOWN:
          Serial.println("Volume DOWN");
          break;

        default:
          Serial.print("Unknown button: 0x");
          Serial.println(code, HEX);
      }
    }

    irrecv.resume();
  }
}

```

Send IR Commands

```
#include <IRremote.h>

#define IR_LED_PIN 3 // IR LED on pin 3 (PWM required)

IRsend irsend;

void setup() {
  Serial.begin(9600);
  Serial.println("IR Transmitter Ready");
}

void loop() {
  if (Serial.available()) {
    char command = Serial.read();

    switch (command) {
      case '1':
        // Send NEC code (Sony TV Power)
        irsend.sendNEC(0x20DF10EF, 32);
        Serial.println("Sent: Power");
        break;

      case '2':
        // Send Sony SIRC code
        irsend.sendSony(0xA90, 12); // Sony TV Power
        Serial.println("Sent: Sony Power");
        break;

      case '3':
        // Send raw timing (recorded from real remote)
        unsigned int rawData[] = {
          9000, 4500, 560, 560, 560, 1690, 560, 560, // ...
        };
        irsend.sendRaw(rawData, sizeof(rawData) / sizeof(rawData[0]), 38);
        Serial.println("Sent: Raw code");
        break;
    }

    delay(100); // Delay between sends
  }
}
```

Learn and Replay Remote Codes

```
#include <IRremote.h>

#define IR_RECV_PIN 11
#define IR_LED_PIN 3

IRrecv irrecv(IR_RECV_PIN);
IRsend irsend;
decode_results results;

#define MAX_BUTTONS 10
unsigned long learnedCodes[MAX_BUTTONS];
int buttonCount = 0;

void setup() {
  Serial.begin(9600);
  irrecv.enableIRin();

  Serial.println("Universal Remote Clone");
  Serial.println("Commands:");
  Serial.println("L - Learn mode");
  Serial.println("1-9 - Replay learned code");
  Serial.println("S - Show learned codes");
}

void loop() {
  // Learn mode
  if (Serial.available()) {
    char cmd = Serial.read();

    if (cmd == 'L' || cmd == 'l') {
      learnButton();
    } else if (cmd >= '1' && cmd <= '9') {
      int index = cmd - '1';
      if (index < buttonCount) {
        replayCode(index);
      }
    } else if (cmd == 'S' || cmd == 's') {
      showCodes();
    }
  }

  // Monitor IR
  if (irrecv.decode(&results)) {
    Serial.print("Detected: 0x");
    Serial.println(results.value, HEX);
    irrecv.resume();
  }
}

void learnButton() {
  Serial.print("Press remote button to learn ");
  Serial.print(buttonCount + 1);
  Serial.println("...");

  irrecv.resume();

  while (true) {
    if (irrecv.decode(&results)) {
      if (results.value != 0xFFFFFFFF) { // Ignore repeats
        learnedCodes[buttonCount] = results.value;
        buttonCount++;

        Serial.print("Learned! Code: 0x");
        Serial.println(results.value, HEX);

        irrecv.resume();
        return;
      }
    }
  }
}

void replayCode(int index) {
  Serial.print("Sending code ");
  Serial.print(index + 1);
  Serial.print(" ");
  Serial.print("0x");
  Serial.println(learnedCodes[index], HEX);

  irsend.sendNEC(learnedCodes[index], 32);
  delay(50);
  irrecv.enableIRin(); // Re-enable receiver after send
}

void showCodes() {
  Serial.println("Learned codes:");
  for (int i = 0; i < buttonCount; i++) {
    Serial.print(i + 1);
    Serial.print(" ");
    Serial.print("0x");
    Serial.println(learnedCodes[i], HEX);
  }
}
```


IR Proximity Sensor



Object Detection with IR

```
#define IR_LED 13      // Regular digital pin
#define IR_RECV A0     // Analog pin (IR phototransistor)

void setup() {
  Serial.begin(9600);
  pinMode(IR_LED, OUTPUT);
  pinMode(IR_RECV, INPUT);
}

void loop() {
  // Turn on IR LED
  digitalWrite(IR_LED, HIGH);
  delay(1); // Brief delay

  // Read reflected IR
  int irValue = analogRead(IR_RECV);

  // Turn off IR LED
  digitalWrite(IR_LED, LOW);

  Serial.print("IR Value: ");
  Serial.print(irValue);

  // Detect object (high value = close object)
  if (irValue > 500) {
    Serial.println(" - Object detected!");
  } else {
    Serial.println(" - No object");
  }

  delay(100);
}

/*
How it works:
1. Turn on IR LED
2. IR light reflects off nearby objects
3. IR phototransistor detects reflected light
4. Close objects = more reflection = higher value

Applications:
- Line-following robots (detect black/white)
- Obstacle detection
- Hand gesture sensing
- Touchless buttons
*/
```